TUGAS KELOMPOK
Java Android App using REST API

NRP & NAMA	:	3223600046 Hari Tammim Mu'Izha
	:	3223600055 Arimbi Kusuma Jati
	:	3223600058 Ihsanta Zaki Sanjaya
KELAS	:	2 D4 Teknik Komputer B
DOSEN	:	Maretha Ruswiansari S.ST., M.T
MATA KULIAH	:	Praktikum Bahasa Pemrograman
TANGGAL	:	Kamis, 17 September 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2024

Tugas Kelompok

Java Android App using REST API

I. Tujuan Pembelajaran

1. Mahasiswa dapat menjelaskan dan menerapkan penggunaan REST API
2. Mahasiswa dapat menjelaskan mengenai penerapan REST API
3. Mahasiswa dapat menjelaskan penerapan REST API

II. Prosedur Percobaan

1. Buka Android Studio, buatlah project baru dengan Empty Views Activity
2. Tambahkan dependencies pada file build.gradle.kts app, seperti berikut:

Build.gradle.kts app

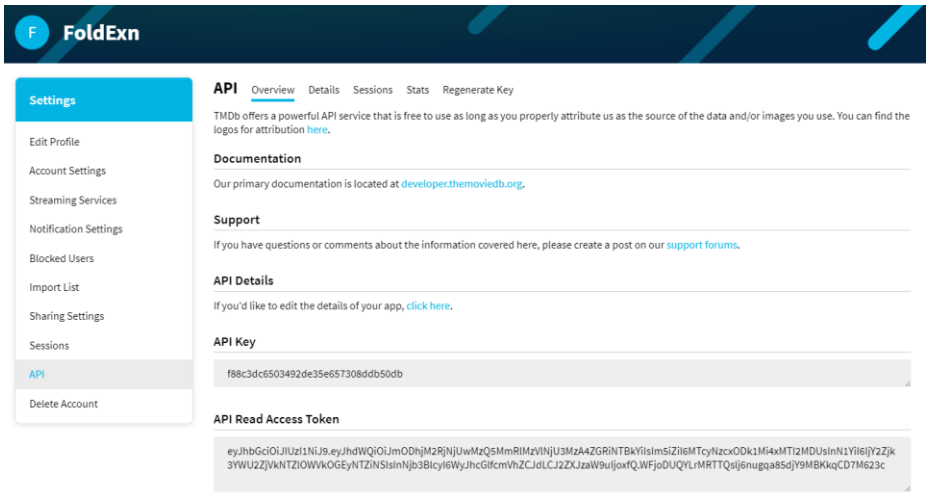
```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.foldexn.foldexm"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.foldexn.foldexm"
        minSdk = 24
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-
optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
}

dependencies {


implementation("androidx.constraintlayout:constraintlayout:2.1.3")
    val lifecycleVersion = "2.4.0"
    implementation("androidx.lifecycle:lifecycle-
viewmodel:$lifecycleVersion")
```

```
    implementation("androidx.lifecycle:lifecycle-
livedata:$lifecycleVersion")

    // Retrofit
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")

    // Glide
    implementation("com.github.bumptech.glide:glide:4.12.0")
    annotationProcessor("com.github.bumptech.glide:compiler:4.12.0")
// Gunakan kapt jika perlu

    implementation("com.google.android.material:material:1.5.0")
    implementation("androidx.recyclerview:recyclerview:1.2.1")
    implementation("androidx.cardview:cardview:1.0.0")
    implementation("androidx.appcompat:appcompat:1.4.1")
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.3")
    androidTestImplementation("androidx.test.espresso:espresso-
core:3.4.0")

    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
}
```

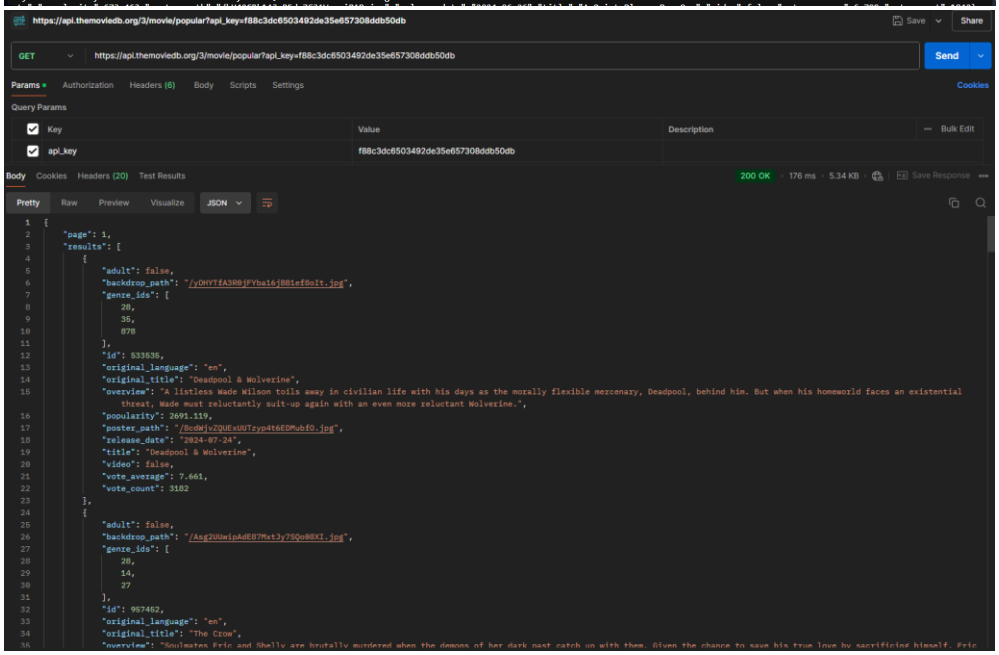
3. Tambahkan internet permission pada AndroidManifest.xml, agar user mengizinkan penggunaan internet.

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />
```

4. Pada aplikasi ini, kita akan membuat movie app, dengan penerapan REST API TMDb. Buat akun tmdb terlebih dahulu, kemudian login. www.themoviedb.org/login
5. Buatlah API Key dengan cara Generate Key pada pengaturan akun TMDb. Catat API Key yang sudah dibuat.



Untuk mengeceknya apakah API bisa diakses, gunakan browser maupun software lain seperti Postman untuk melihat apakah request berhasil.



6. Buatlah Credentials.java yang akan digunakan untuk mengakses API The Movie Database (TMDb) berdasarkan url sebelumnya.

```
public class Credentials {
    public static final String BASE_URL =
"https://api.themoviedb.org/3/";
    public static final String KEY =
"f88c3dc6503492de35e657308ddb50db";
    public static final String Poster_URL =
"https://image.tmdb.org/t/p/w500/";
}
```

7. Selanjutnya buatlah AppExecutor untuk mengatur thread untuk tugas-tugas yang dijalankan secara asinkron, sehingga aplikasi dapat melakukan panggilan jaringan tanpa memblokir UI di latar belakang, seperti berikut:

AppExecutor

```
package com.utils;

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;

public class AppExecutor {
    private static AppExecutor instance;
    public static AppExecutor getInstance() {
        if (instance == null) {
            instance = new AppExecutor();
        }
        return instance;
    }

    private final ScheduledExecutorService networkIO =
Executors.newScheduledThreadPool(3);
    public ScheduledExecutorService getNetworkIO() {
        return networkIO;
    }
}
```

8. Lalu, kita buat interface MovieApi yang mendefinisikan endpoint API dan methodsnya, seperti berikut:

MovieApi

```
package com.utils;

import com.response.PopularMovieResponses;
import com.response.SearchMovieResponses;

import retrofit2.Call;
import retrofit2.http.GET;
import retrofit2.http.Query;

public interface MovieApi {
    @GET("movie/popular")
        Call<PopularMovieResponses> getPopularMovie
(@Query("api_key") String key,
                                                    @Query("page")
int page);
    // earch movie
    @GET("search/movie")
    Call<SearchMovieResponses> searchMovie(@Query("api_key") String
key,
                                           @Query("query") String
query,
                                           @Query("page") int page);
}
```

9. Kemudian buatlah kelas dalam paket request yang berfungsi untuk mengelola permintaan ke API menggunakan Retrofit, seperti berikut:

ApiService

```
package com.request;

import com.utils.Credentials;
import com.utils.MovieApi;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class ApiService {
    private static Retrofit.Builder retrofitBuilder = new
Retrofit.Builder().baseUrl(Credentials.BASE_URL).addConverterFactory(
GsonConverterFactory.create());
    private static Retrofit retrofit = retrofitBuilder.build();
    private static MovieApi movieApi =
retrofit.create(MovieApi.class);

    public static MovieApi getMovieApi() {

        return movieApi;
    }
}
```

Popular…request

```
package com.request;

import android.util.Log;

import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;

import com.models.MovieModel;
import com.response.PopularMovieResponses;
import com.utils.AppExecutor;
import com.utils.Credentials;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

import retrofit2.Call;
import retrofit2.Response;

public class PopularMovieApiClient {
    private static PopularMovieApiClient instance;
    public static PopularMovieApiClient getInstance() {
        if (instance == null) {
            instance = new PopularMovieApiClient();
        }
            return instance;
    }

    private final MutableLiveData<List<MovieModel>>
popularMovieLiveData;
    private PopularRunnable popularRunnable;
    private PopularMovieApiClient() {
        popularMovieLiveData = new MutableLiveData<>();
    }
    public LiveData<List<MovieModel>> getPopularMovie() {
```

```
        return popularMovieLiveData;
    }

    public void getPopularMovie(int page) {
        if (popularRunnable != null) {
            popularRunnable = null;
        }
        popularRunnable = new PopularRunnable(page);

        final Future handler =
AppExecutor.getInstance().getNetworkIO().submit(popularRunnable);
        AppExecutor.getInstance().getNetworkIO().schedule(() -> {
            handler.cancel(true);
        }, 3000, TimeUnit.MILLISECONDS);
    }

    private class PopularRunnable implements Runnable {
        private final int page;
        boolean cancleRequest;
        public PopularRunnable(int page) {
            this.page = page;
            cancleRequest = false;
        }

        @Override
        public void run() {
            //get req
            try {
                Response<PopularMovieResponses> response =
getPopularMovie(page).execute();
                if (cancleRequest) {
                    return;
                }
                if (response.isSuccessful()) {
                    if (response.code() == 200) {
                        assert response.body() != null;
                        List<MovieModel> popularMovieList = new
ArrayList<>(((PopularMovieResponses) response.body()).getMovies());

                        // Log
                        Log.d("PopularMovieApiClient", "Received
movies: " + popularMovieList.size());

                        if (page == 1) {

popularMovieLiveData.postValue(popularMovieList);
                        } else {
                            List<MovieModel> currentMovie =
popularMovieLiveData.getValue();
                            assert currentMovie != null;
                            currentMovie.addAll(popularMovieList);

popularMovieLiveData.postValue(currentMovie);
                        }
                    } else {
                        Log.e("PopularMovieApiClient", "Error: " +
response.errorBody().string());
                        popularMovieLiveData.postValue(null);
                    }
                } else {
                    Log.e("PopularMovieApiClient", "Request isn't
```

```
successful");
                }
            } catch (IOException e) {
                Log.e("PopularMovieApiClient", "IOException: " + e);
                popularMovieLiveData.postValue(null);
            }
        }
        private Call<PopularMovieResponses> getPopularMovie(int page)
{
            return
ApiService.getMovieApi().getPopularMovie(Credentials.KEY,page);

        }
    }
}
```

Search…request

```
package com.request;

import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;

import com.models.MovieModel;
import com.response.SearchMovieResponses;
import com.utils.AppExecutor;
import com.utils.Credentials;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

import retrofit2.Call;
import retrofit2.Response;

public class SearchMovieApiClient {
    private static SearchMovieApiClient instance;

    public static SearchMovieApiClient getInstance() {
        if (instance == null) {
            instance = new SearchMovieApiClient();
        }
        return instance;
    }

    // LiveData
    private final MutableLiveData<List<MovieModel>>
searchMovieLiveData;
    //runnable
    private SearchRunnable searchRunnable;
    private SearchMovieApiClient() {
        searchMovieLiveData = new MutableLiveData<>();
    }
    public LiveData<List<MovieModel>> getSearchMovie() {
        return searchMovieLiveData;
    }
    public void getSearchMovie(String query, int page) {
        if (searchRunnable != null) {
```

```
            searchRunnable = null;
        }
        searchRunnable = new SearchRunnable(query, page);

        final Future<?> handler =
AppExecutor.getInstance().getNetworkIO().submit(searchRunnable);
        AppExecutor.getInstance().getNetworkIO().schedule(() -> {
            // Membatalkan
            handler.cancel(true);
        }, 3000, TimeUnit.MILLISECONDS);
    }

    // Retrieve data from REST API using runnable
    private class SearchRunnable implements Runnable {
        private String query;
        private final int page;
        boolean cancleRequest;

        public SearchRunnable(String query, int page) {
            this.query = query;
            this.page = page;
            cancleRequest = false;
        }

        @Override
        public void run() {
            try {
                // query
                if (query == null || query.trim().isEmpty()) {
                    System.out.println("Query is empty");
                    searchMovieLiveData.postValue(null);
                    return;
                }

                System.out.println("Searching for query: " + query);

                // take respons
                Response<SearchMovieResponses> response =
searchMovie(query, page).execute();

                if (cancleRequest) {
                    return;
                }

                if (response.isSuccessful()) {
                    if (response.code() == 200) {
                        assert response.body() != null;
                        List<MovieModel> searchMovieList = new
ArrayList<>(((SearchMovieResponses) response.body()).getMovies());
                        if (page == 1) {

searchMovieLiveData.postValue(searchMovieList);
                        } else {
                            List<MovieModel> currentMovie =
searchMovieLiveData.getValue();
                            assert currentMovie != null;
                            currentMovie.addAll(searchMovieList);

searchMovieLiveData.postValue(currentMovie);
                        }
                    }
```

```
                } else {
                    // Logging err
                    int statusCode = response.code();
                    System.out.println("Error Code: " + statusCode);
                    if (response.errorBody() != null) {
                        System.out.println("Error Body: " +
response.errorBody().string());
                    }
                    searchMovieLiveData.postValue(null);
                }
            } catch (IOException e) {
                System.out.println("Network error: " +
e.getMessage());
                searchMovieLiveData.postValue(null);
            }
        }


        private Call<SearchMovieResponses> searchMovie(String query,
int page) {
            Call<SearchMovieResponses> call =
ApiService.getMovieApi().searchMovie(Credentials.KEY, query, page);
            System.out.println("Calling API URL: " +
call.request().url()); // Logging URL
            return call;
        }
    }
}
```

10. Buatlah model respons untuk API TMDb dengan menambahkan class baru. Class ini berfungsi untuk mendeserialisasi data JSON yang diterima API dan mengelola daftar film yang diterima dalam bentuk objek, seperti berikut:

    SearchMovieResponses

```
package com.response;

import com.models.MovieModel;
import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;

import java.util.List;

public class SearchMovieResponses {
    @SerializedName("results")
    @Expose
    private List<MovieModel> movies;

    public List<MovieModel> getMovies() {
        return movies;
    }

    @Override
    public String toString() {
        return "SearchMovieResponses{" +
                "movies=" + movies +
                '}';
    }
}
```

PopularMovieResponses

```
package com.response;

import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;
import com.models.MovieModel;

import java.util.List;

public class PopularMovieResponses {
    @SerializedName("results")
    @Expose
    private List<MovieModel> movies;

    public List<MovieModel> getMovies() {
        return movies;
    }

    @Override
    public String toString() {
        return "MovieResponses{" +
                "movies=" + movies +
                '}';
    }
}
```

11. Selanjutnya buatlah repository, yang bertindak sebagai penghubung antara data dan UI, seperti berikut:

Popular…repo

```
package com.response;

import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;
import com.models.MovieModel;

import java.util.List;

public class PopularMovieResponses {
    @SerializedName("results")
    @Expose
    private List<MovieModel> movies;

    public List<MovieModel> getMovies() {
        return movies;
    }
    @Override
    public String toString() {
        return "MovieResponses{" +
                "movies=" + movies +
                '}';
    }
}
```

Search…repo

```
package com.response;

import com.models.MovieModel;
import com.google.gson.annotations.Expose;
```

```
import com.google.gson.annotations.SerializedName;

import java.util.List;

public class SearchMovieResponses {
    @SerializedName("results")
    @Expose
    private List<MovieModel> movies;
    public List<MovieModel> getMovies() {
        return movies;
    }

    @Override
    public String toString() {
        return "SearchMovieResponses{" +
                "movies=" + movies +
                '}';
    }
}
```

12. Buatlah kelas untuk model, yang berperan dalam arsitektur aplikasi, menyimpan dan mengelola data terkait UI, dan penghubung repository dan layout, seperti berikut:

MovieModel

```
package com.models;

import android.os.Parcel;
import android.os.Parcelable;
import androidx.annotation.NonNull;

public class MovieModel implements Parcelable {
    private int id;
    private String title;
    private String poster_path;
    private String release_date;
    private float vote_average;
    private String overview;
    private String original_language;

    public MovieModel(int id, String title, String poster_path,
String release_date, float vote_average, String overview, String
original_language) {
        this.id = id;
        this.title = title;
        this.poster_path = poster_path;
        this.release_date = release_date;
        this.vote_average = vote_average;
        this.overview = overview;
        this.original_language = original_language;
    }

    protected MovieModel(Parcel in) {
        id = in.readInt();
        title = in.readString();
        poster_path = in.readString();
        release_date = in.readString();
        vote_average = in.readFloat();
        overview = in.readString();
```

```
        original_language = in.readString();
    }

    public static final Creator<MovieModel> CREATOR = new
Creator<MovieModel>() {
        @Override
        public MovieModel createFromParcel(Parcel in) {
            return new MovieModel(in);
        }

        @Override
        public MovieModel[] newArray(int size) {
            return new MovieModel[size];
        }
    };

    public int getId() {
        return id;
    }
    public String getTitle() {
        return title;
    }
    public String getPoster_path() {
        return poster_path;
    }
    public String getRelease_date() {
        return release_date;
    }
    public float getVote_average() {
        return vote_average;
    }
    public String getOverview() {
        return overview;
    }
    public String getOriginal_language() {
        return original_language;
    }

    @Override
    public int describeContents() {
        return 0;
    }

    @Override
    public void writeToParcel(@NonNull Parcel parcel, int i) {
        parcel.writeInt(id);
        parcel.writeString(title);
        parcel.writeString(poster_path);
        parcel.writeString(release_date);
        parcel.writeFloat(vote_average);
        parcel.writeString(overview);
        parcel.writeString(original_language);
    }

    @Override
    public String toString() {
        return "MovieModel{" +
                "id=" + id +
                ", title='" + title + '\'' +
                ", poster_path='" + poster_path + '\'' +
                ", release_date='" + release_date + '\'' +
```

```
                ", vote_average=" + vote_average +
                ", overview='" + overview + '\'' +
                ", original_language='" + original_language + '\'' +
                '}';
    }
}
```

Popular..Model

```
package com.models.viewmodels;

import androidx.lifecycle.LiveData;
import androidx.lifecycle.ViewModel;

import com.models.MovieModel;
import com.repositories.PopularMovieRepository;

import java.util.List;

public class PopularMovieListViewModel extends ViewModel {
    private PopularMovieRepository popularMovieRepository;
    public  PopularMovieListViewModel() {
        popularMovieRepository =
PopularMovieRepository.getInstance();
    }

    public LiveData<List<MovieModel>> getPopularMovie() {
        return popularMovieRepository.getPopularMovie();
    }

    public void getPopularMovie(int page) {
        popularMovieRepository.getPopularMovie(page);
    }

    public void popularMovieNextPage() {
        popularMovieRepository.popularMovieNextPage();
    }
}
```

Search…Model

```
package com.models.viewmodels;
import androidx.lifecycle.LiveData;
import androidx.lifecycle.ViewModel;

import com.models.MovieModel;
import com.repositories.PopularMovieRepository;
import com.repositories.SearchMovieRepository;

import java.util.List;

public class SearchMovieListViewModel extends ViewModel {
    private SearchMovieRepository searchMovieRepository;

    public SearchMovieListViewModel() {
        searchMovieRepository = SearchMovieRepository.getInstance();
    }
    public LiveData<List<MovieModel>> getSearchMovie() {
        return searchMovieRepository.getSearchMovie();
    }
    public void getSearchMovie(String query,int page) {
        searchMovieRepository.getSearchMovie(query,page);
```

```
    }

    // next page
    public void searchMovieNextPage() {
        searchMovieRepository.searchMovieNextPage();

    }
}
```

13. Tak lupa untuk membuat layout untuk menampilkan daftar film dan detail dari film yang ada, seperti berikut:

activity_detail_movie

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="?attr/bg"
    tools:context=".DetailMovieActivity">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <RelativeLayout
                android:layout_width="match_parent"
                android:layout_height="550dp">

                <ImageView
                    android:id="@+id/posterDetail"
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:contentDescription="@string/todo"
                    android:scaleType="centerCrop"
                    tools:src="@tools:sample/avatars" />

            </RelativeLayout>

            <RelativeLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content">

                <TextView
                    android:id="@+id/titleDetail"
                    android:layout_width="220dp"
                    android:layout_height="wrap_content"
                    android:layout_marginStart="30dp"
                    android:layout_marginTop="20dp"
                    android:layout_marginEnd="30dp"
                    android:fontFamily="@font/poppins_semi_bold"
                    android:textColor="?attr/boldText"
```

```
            android:textSize="28sp"
            tools:text="The Irregular" />

        <TextView
            android:id="@+id/releaseDateDetail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"
            android:layout_marginTop="28dp"
            android:layout_marginEnd="30dp"
            android:fontFamily="@font/poppins_medium"
            tools:text="23 April2021"
            android:textColor="?attr/boldText"
            android:textSize="18sp" />

        <TextView
            android:id="@+id/ratingTextDetail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@+id/titleDetail"
            android:layout_alignParentEnd="true"
            android:layout_marginTop="18dp"
            android:layout_marginEnd="30dp"
            android:fontFamily="@font/poppins_semi_bold"
            android:textColor="@color/material_blue"
            android:textSize="14sp"
            tools:text="8.5" />

        <LinearLayout
            android:id="@+id/dotsDetail"
            android:layout_marginTop="-4dp"
            android:layout_marginEnd="8dp"
            android:layout_below="@+id/titleDetail"
            android:layout_toStartOf="@+id/ratingTextDetail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="horizontal" />

        <TextView
            android:id="@+id/overviewTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@+id/titleDetail"
            android:layout_marginStart="30dp"
            android:layout_marginTop="20dp"
            android:layout_marginEnd="30dp"
            android:fontFamily="@font/poppins_semi_bold"
            android:text="@string/overview"
            android:textColor="@color/material_blue"
            android:textSize="18sp" />

        <TextView
            android:id="@+id/overview"
            android:layout_below="@+id/overviewTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:layout_marginStart="30dp"
            android:layout_marginEnd="30dp"
            android:layout_marginBottom="30dp"
            android:fontFamily="@font/poppins_regular"
```

```
                    android:textColor="?attr/regularText"
                    android:textSize="14sp"
                    android:text="@string/overview"/>


            </RelativeLayout>

        </LinearLayout>

    </ScrollView>

</LinearLayout>
```

activity_movie_list

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="?attr/bg"
    tools:context="com.MovieListActivity">
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="10dp"
        app:layout_scrollFlags="scroll|enterAlways"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:theme="@style/ThemeOverlay.AppCompat.ActionBar"
        app:popupTheme="@style/ThemeOverlay.AppCompat.ActionBar"
        android:background="?attr/bg">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:fontFamily="@font/poppins_semi_bold"
            android:textSize="24sp"
            android:text="@string/movieku"
            android:textColor="@color/material_blue"/>

        <androidx.appcompat.widget.SearchView
            android:layout_width="match_parent"
            android:layout_height="40dp"
            android:id="@+id/searchView"
            app:queryHint="Search Movie"
            android:background="@drawable/rounded_searchbar"
            android:layout_marginStart="50dp"
            android:layout_marginEnd="30dp"/>
    </androidx.appcompat.widget.Toolbar>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycleView"
        android:layout_width="match_parent"
        android:layout_height="0dp"

app:layout_behavior="com.google.android.material.appbar.AppBarLayout$
```

```
ScrollinngViewBehavior"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/toolbar"
        app:layout_constraintVertical_bias="1.0"
        tools:listitem="@layout/movie_list" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

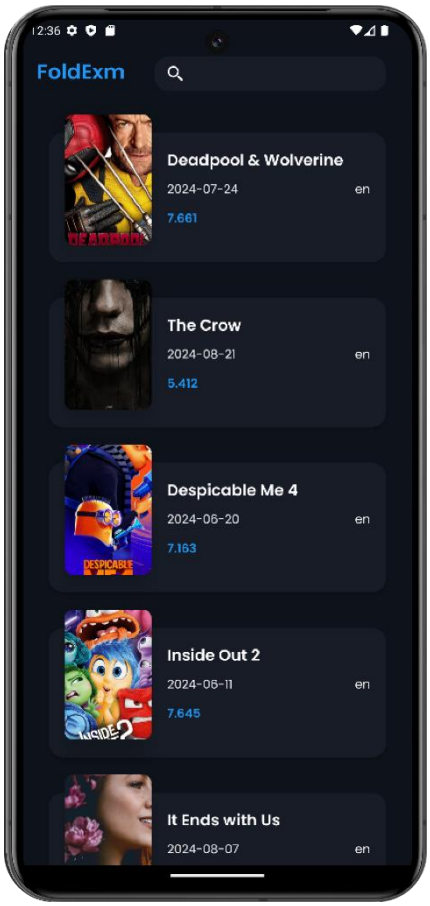
movie_list

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="190dp">
    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="149dp"
        android:layout_marginTop="30dp"
        android:layout_marginStart="30dp"
        android:layout_marginEnd="30dp"
        android:paddingStart="136dp"
        android:paddingEnd="18dp"
        android:paddingTop="18dp"
        android:elevation="2dp"
        android:background="@drawable/card_layout">
        <TextView
            android:id="@+id/title"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:fontFamily="@font/poppins_semi_bold"
            android:textColor="?attr/boldText"
            android:textSize="18sp"
            tools:text="The Irregular" />
        <TextView
            android:id="@+id/releaseDate"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@+id/title"
            android:layout_marginTop="6dp"
            android:fontFamily="@font/poppins_regular"
            tools:text="23 April 2021"
            android:textColor="?attr/regularText"
            android:textSize="14sp" />

        <TextView
            android:id="@+id/lang"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentEnd="true"
            android:layout_below="@+id/title"
            android:layout_marginTop="6dp"
            tools:text="Eng"
            android:fontFamily="@font/poppins_regular"
            android:textColor="?attr/regularText"
            android:textSize="14sp" />

        <LinearLayout
```

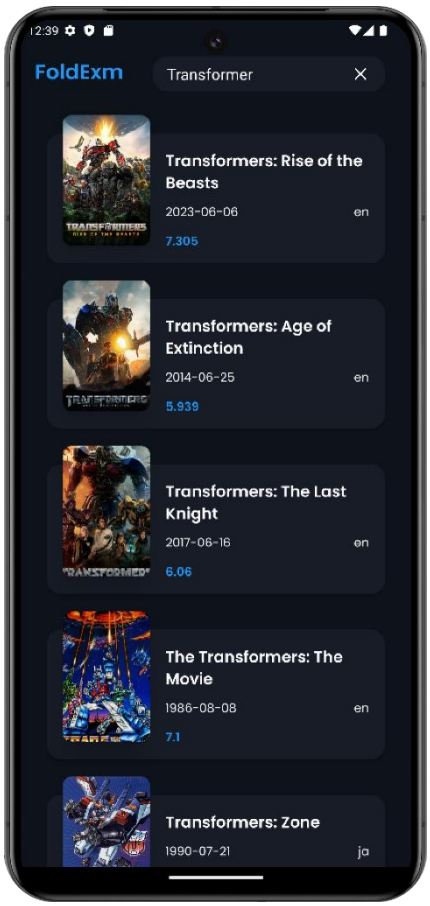
```
            android:id="@+id/dots"
            android:layout_marginTop="69dp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="horizontal" />

        <TextView
            android:id="@+id/ratingText"
            android:layout_below="@+id/releaseDate"
            android:layout_toEndOf="@+id/dots"
            android:layout_marginTop="10dp"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            tools:text="8.5"
            android:fontFamily="@font/poppins_semi_bold"
            android:textSize="14sp"
            android:textColor="@color/material_blue"/>

    </RelativeLayout>

    <androidx.cardview.widget.CardView
        android:id="@+id/posterViewModel"
        android:layout_width="100dp"
        android:layout_height="150dp"
        android:layout_marginStart="48dp"
        android:layout_marginTop="-170dp"
        app:cardCornerRadius="10dp"
        app:cardElevation="15dp">

        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent">

            <ImageView
                android:id="@+id/poster"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:scaleType="centerCrop"
                tools:src="@tools:sample/avatars"
                android:contentDescription="@string/todo"/>
        </RelativeLayout>

    </androidx.cardview.widget.CardView>

</LinearLayout>
```

14. Kemudian sesuaikan values dan themes, values digunakan untuk mendefinisikan sumber daya seperti string, warna, dan dimensi yang konsisten di seluruh antarmuka pengguna. Selain itu, tema yang diterapkan dalam aplikasi menciptakan tampilan yang seragam dan menarik,

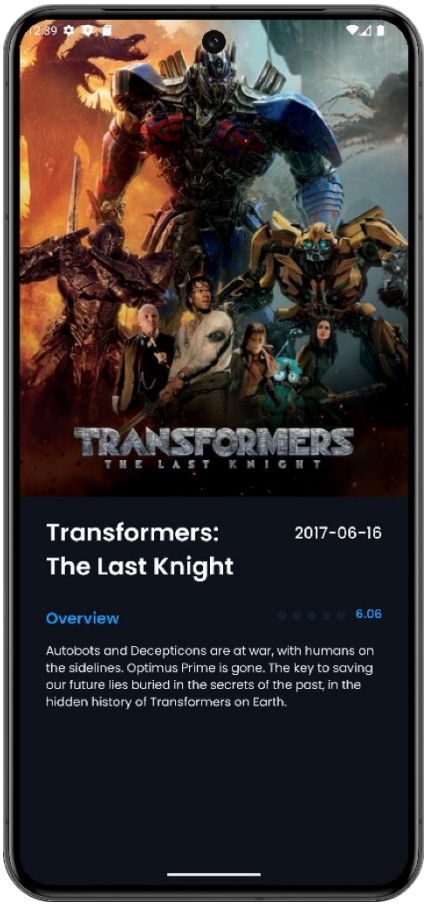
III. Hasil Percobaan



Tampilan Utama



Tampilan Setelah Pencarian



Tampilan Detail Film

IV. Analisa

Dapat di analisa pada Praktikum 6 yaitu praktikum untuk mengambil data dari sebuah database menggunakan fitur REST API. Cara kerja aplikasi ini adalah mencari website yang menyediakan API, lalu jika di beri “/ ...” maka akan memberi tampilan JSON, JSON untuk mengolah data yang nantinya di olah di android, pada data JSON di ambil string nya dan akan di munculkan ke textview, jadi mengirim pada website, lalu website ngirim data dalam bentuk GSON, data GSON di ambil string nya lalu di olah pada android studio untuk fungsi.

Untuk menargetkan “id” yang di ambil dari data GSON, menggunakan kode

```
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.9.0")
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

```
public class Credentials {  10 usages
    public static final String BASE_URL = "https://api.themoviedb.org/3/";  1 usage
    public static final String KEY = "f88c3dc6503492de35e657308ddb50db";  2 usages
    public static final String Poster_URL = "https://image.tmdb.org/t/p/w500/";  2 usages
}
```

```
public class MovieModel implements Parcelable {
    private int id;  5 usages
    private String title;  5 usages
    private String poster_path;  5 usages
    private String release_date;  5 usages
    private float vote_average;  5 usages
    private String overview;  5 usages
    private String original_language;  5 usages
```

program berikut :

```
package com.request;

import com.utils.Credentials;
import com.utils.MovieApi;

import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class ApiService {  2 usages
    private static Retrofit.Builder retrofitBuilder = new Retrofit.Builder().baseUrl(Credentials.BASE_URL).addConverterFactory(GsonConvert
    private static Retrofit retrofit = retrofitBuilder.build();  1 usage
    private static MovieApi movieApi = retrofit.create(MovieApi.class);  1 usage

    public static MovieApi getMovieApi() {  2 usages

        return movieApi;
    }
}
```

Setelah di targetkan, untuk menampilkan di cardView harus menunggu respon dari website yang tadi di gunakan, jika website belum selesai merespondata yang di inginkan dan android tidak bisa menunggu respon sampai proses selesai, maka data yang di tampilkan di cardView kosong. Jadi intinya data di minta atau request data lalu website ngirim data.

Terdapat Volley berfungsi untuk mempermudah dan, yang terpenting, mempercepat networking untukaplikasi Android. Volley terintegrasi secara mudah dengan protokol apa pun dan memberi performa terbaik dengan dukungan untuk string raw, gambar, dan GSON.

Pada file.java tambahkan library yang berfungsi untuk menggunakan fitur REST API pada android dan tambahkan library GSON untuk mengolah data GSON pada APIyang akan digunakan, ini tadi sudah di jelaskan di atas:

```
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;
```

```
import retrofit2.Call;
import retrofit2.Response;
```

```
import com.google.gson.annotations.Expose;
import com.google.gson.annotations.SerializedName;
```

Untuk membuat sebuah Request atau yang di inginkan pada API tambahkan kode berikut:

```
private Call<SearchMovieResponses> searchMovie(String query, int page) {  1 usage
    Call<SearchMovieResponses> call = ApiService.getMovieApi().searchMovie(Credentials.KEY, query, page);
    System.out.println("Calling API URL: " + call.request().url()); // Logging URL yang akan dipanggil
    return call;
}
```

```
private Call<PopularMovieResponses> getPopularMovie(int page) {  1 usage
    return ApiService.getMovieApi().getPopularMovie(Credentials.KEY,page);

}
```

Kemudian untuk mengambil data JSON pada API gunakan kode berikut:

```
private final MutableLiveData<List<MovieModel>> popularMovieLiveData;  7 usages

private PopularRunnable popularRunnable;  4 usages

private PopularMovieApiClient() {  1 usage
    popularMovieLiveData = new MutableLiveData<>();
}
public LiveData<List<MovieModel>> getPopularMovie() { return popularMovieLiveData; }
```

```
private final MutableLiveData<List<MovieModel>> searchMovieLiveData;  8 usages

// Variabel global untuk runnable
private SearchRunnable searchRunnable;  4 usages

private SearchMovieApiClient() { searchMovieLiveData = new MutableLiveData<>(); }

public LiveData<List<MovieModel>> getSearchMovie() { return searchMovieLiveData; }
```

Kode diatas berfungsi untuk

Fungsi getPopularMovie(int page) bertujuan untuk membuat permintaan ke API guna mendapatkan daftar film populer berdasarkan nomor halaman yang diberikan. Fungsi ini memanggil metode dalam antarmuka API yang telah didefinisikan menggunakan Retrofit, dengan menggunakan Credentials.KEY sebagai kunci API untuk autentikasi. Fungsi ini mengembalikan objek Call<PopularMovieResponses>, yang dapat dieksekusi untuk melakukan permintaan ke server.

Sementara itu, fungsi searchMovie(String query, int page) digunakan untuk mencari film berdasarkan kata kunci yang dimasukkan oleh pengguna. Fungsi ini juga memanggil metode di dalam antarmuka API menggunakan Retrofit, dengan Credentials.KEY, query pencarian, dan nomor halaman sebagai parameter. Selain itu, fungsi ini mencetak URL yang akan dipanggil ke konsol untuk tujuan debugging. Seperti fungsi sebelumnya, searchMovie mengembalikan objek Call<SearchMovieResponses>, yang memungkinkan eksekusi permintaan pencarian film.

Dari data yang telah di ambil dari REST API dengan program tersebut maka data dari REST API tersebut akan di tampilkan pada layout yang telah di buat menggunakan program berikut

```
private void ObservasingAnyChangesPopularMovie() {  1 usage
    popularMovieListViewModel.getPopularMovie().observe( owner: this, movieModels -> {
        // observing any data changes
        if (movieModels != null) {
            for (MovieModel model : movieModels) {
                // get data
                recycleViewAdapter.setMovie(movieModels);
            }
        }
    });
}

private void ObservasingAnyChangesSearchMovie() {  1 usage
    searchMovieListViewModel.getSearchMovie().observe( owner: this, movieModels -> {
        // observing any data changes
        if (movieModels != null) {
            for (MovieModel model : movieModels) {
                // get data
                recycleViewAdapter.setMovie(movieModels);
            }
        }
    });
}
```

```
private void getMovieDetail() {  1 usage
    if (getIntent().hasExtra( name: "movie")) {
        MovieModel model = getIntent().getParcelableExtra( name: "movie");

        // string text
        String getRating = String.valueOf(model.getVote_average());
        TextView[] dots;
        int dotSize = 5;

        title.setText(model.getTitle());
      ((MovieViewHolder)holder).tagline.setText(movieList.get(position).get());
        releaseDate.setText(model.getRelease_date());
        overview.setText(model.getOverview());
        rating.setText(getRating);

        // poster
        Glide.with( activity: this).load( string: Credentials.Poster_URL + model.getPoster_path()).into(poster);

        // set dots rating
        dots = new TextView[dotSize];
        float ratings = model.getVote_average();
        dotsLayout.removeAllViews();

        for (int j = 0; j < dots.length; j++) {
```

Dalam kode tersebut, terdapat dua metode yang berfungsi untuk mengamati perubahan data dari ViewModel: Observasing AnyChangesPopularMovie dan ObservasingAnyChangesSearchMovie. Metode ObservasingAnyChangesPopularMovie mengamati objek LiveData yang dikembalikan oleh popularMovieListViewModel.getPopularMovie(). Ketika data film populer (movieModels) diperoleh, metode ini akan memeriksa apakah data tersebut tidak null. Jika data tidak null, ia akan iterasi melalui setiap objek MovieModel dalam movieModels dan memanggil metode setMovie (movieModels) pada recycleViewAdapter, yang bertujuan

untuk memperbarui tampilan RecyclerView dengan data film yang baru diambil. Proses serupa dilakukan dalam metode Observasing AnyChangesSearchMovie, yang mengamati hasil pencarian film melalui objek LiveData dari searchMovieListViewModel. Dengan demikian, kedua metode ini memastikan bahwa tampilan di layout selalu diperbarui sesuai dengan data terbaru yang diperoleh dari API, baik untuk film populer maupun hasil pencarian.

Fungsi getMovieDetail() bertanggung jawab untuk menampilkan detail film yang dipilih oleh pengguna di layar. Pertama, fungsi ini memeriksa apakah intent yang diterima memiliki ekstra bernama "movie". Jika ada, fungsi ini akan mengambil objek MovieModel dari intent tersebut. Setelah itu, informasi seperti judul, tanggal rilis, ringkasan, dan rating film diambil dari objek model dan ditampilkan pada elemen UI yang sesuai, seperti TextView. Gambar poster film diunduh menggunakan pustaka Glide dan dimuat ke dalam tampilan poster. Fungsi ini juga mengatur tampilan rating dalam bentuk titik (dots) yang diwarnai berdasarkan nilai rating film, dengan lima titik yang mewakili skala 10. Proses pewarnaan titik dilakukan dengan beberapa loop yang memeriksa nilai rating dan mengubah warna titik sesuai dengan rentang rating yang diberikan. Dengan demikian, fungsi ini menyajikan informasi lengkap tentang film kepada pengguna dengan cara yang terstruktur dan menarik.

V. Kesimpulan

Dapat di simpulkan pada praktikum Rest Api bahwa, REST API merupakan penerapan dari API (Application Programming Interface). Sedangkan REST (Representional State Transfer) adalah sebuah arsitektur metode komunikasi yang menggunakan protokol HTTP untuk pertukaran data dimana metode ini sering diterapkan dalam pengembangan aplikasi.

REST API adalah teknologi yang memudahkan para web developer jaman now sebagai penyedia resource (sumber data/daya) yang diperuntukan untuk client, bisa itu android maupun IOS. Teknologi REST API membantu dalam proses pengambilan data dari server dan memungkinkan data terbarui secara realtime. Teknologi REST API mudah dibaca oleh pengembang karena data yang dihasilkan berupa data format JSON.

Ada dua hal yang perlu kita ketahui mengenai RESTful API yaitu setiap URL yang ada pada RESTful API disebut dengan Request sedangkan response adalah data yang dikembalikan setelah client melakukan request.